CONTRACT NOTE REWORK

DocuSign Integration

Estimate 2 - Technical Walkthrough

~7.1 developer days (4.1 required + testing)

D55 Consulting

July 2026

BrytEnergy

Background

Once a contract note PDF has been produced, it still needs to be signed by the customer. Today that final step is manual: someone has to send the document out, chase the signature, collect the signed copy, and file it against the customer's record.

Estimate 2 automates that entire loop. When Estimate 1's render pipeline writes a finished PDF, this system automatically sends it to the customer for electronic signature via DocuSign, then retrieves the signed copy and attaches it to the customer's Salesforce record.

It is a headless, event-driven pipeline. There is no new screen in the Admin Portal, the whole thing runs automatically from the moment a PDF lands. The only visible outcome for the business is a signed contract appearing against the customer in Salesforce.

> ### Builds directly on Estimate 1
>
> This estimate consumes the output of Estimate 1. The trigger is a contract note PDF landing in the S3 output bucket. Estimate 1 is extended slightly to write a small metadata sidecar alongside each PDF (customer reference, offer reference, name) so this pipeline knows who to send it to.

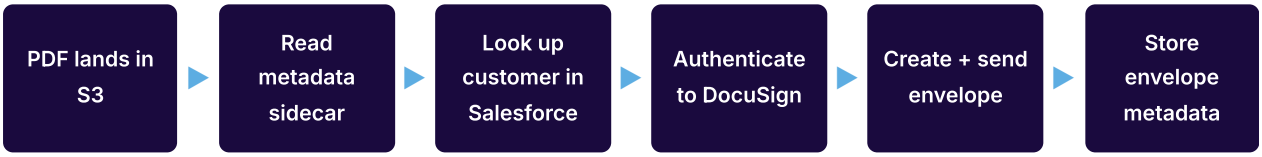
How it works

The system is two small Lambda functions with DocuSign and Salesforce either side of them.

The first Lambda runs when a PDF is produced: it looks up the customer in Salesforce, authenticates to DocuSign, and creates a signing envelope, which prompts DocuSign to email the customer. The second Lambda runs when the customer finishes signing: DocuSign calls back over a webhook, and the Lambda downloads the signed PDF, stores it, and attaches it to Salesforce.

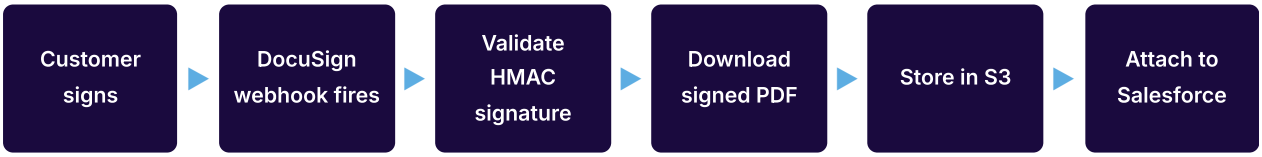
In between those two moments, the customer signs at their own pace, so the flow is split into a send phase and a completion phase rather than one continuous run.

Send phase (automatic, on PDF creation)



DocuSign emails the signing request to the customer as soon as the envelope is sent.

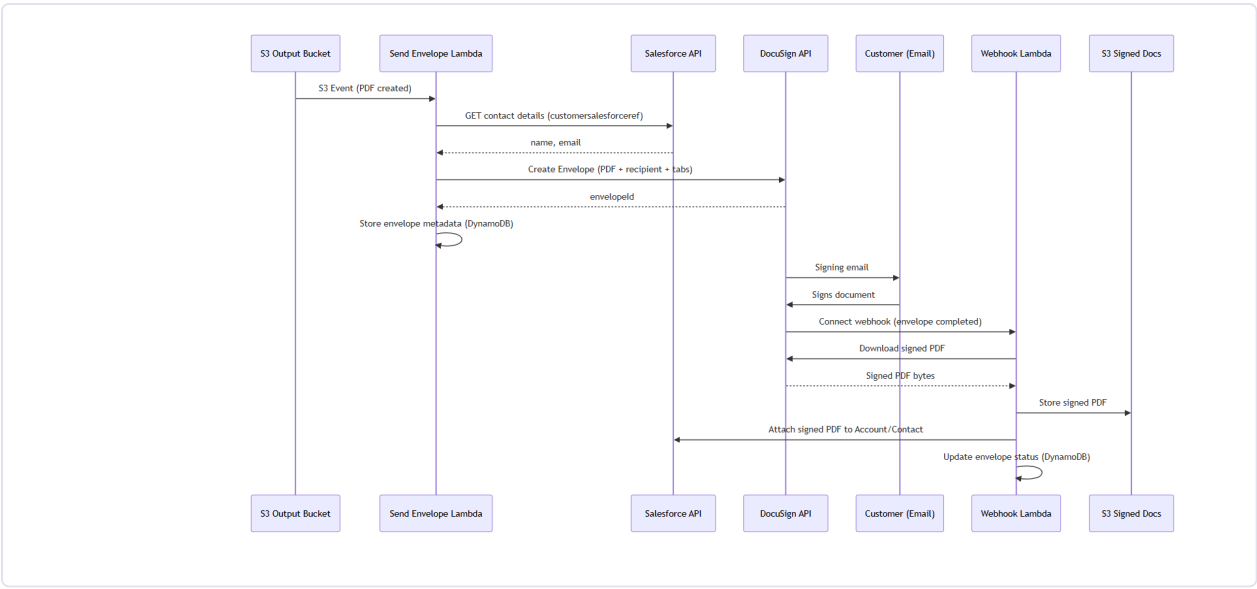
Completion phase (triggered when the customer signs)



A declined or expired envelope instead writes a notification record for operational follow-up.

End-to-end sequence

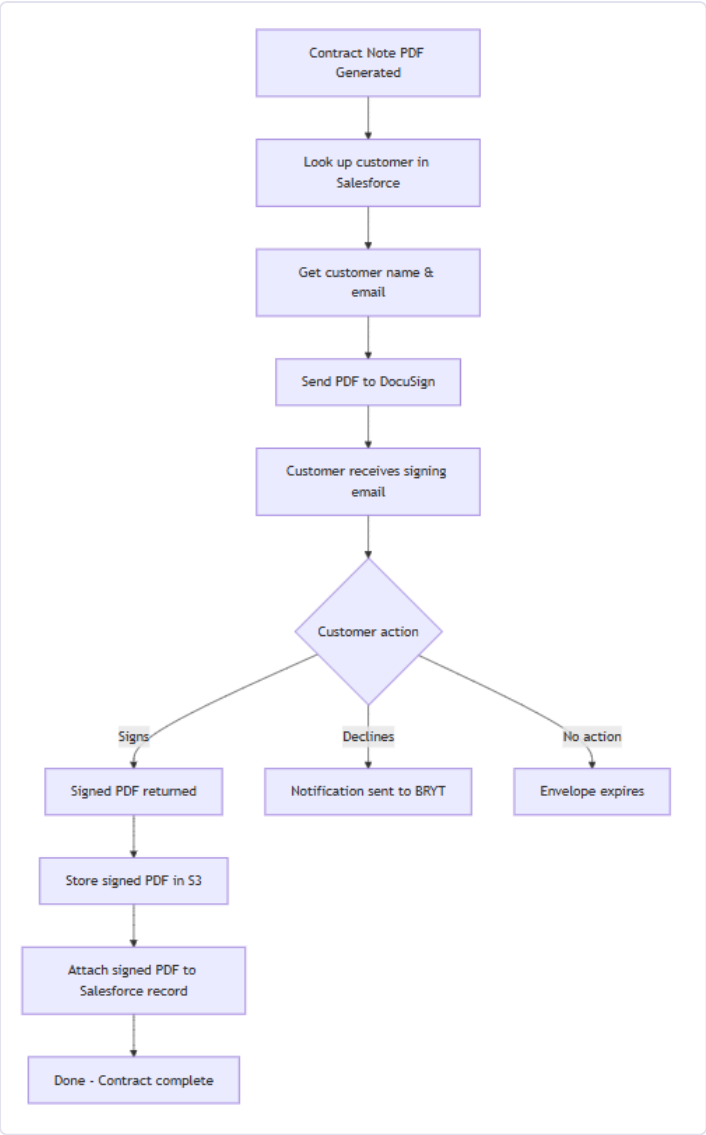
The full exchange between the render pipeline, the two Lambdas, DocuSign, the customer, and Salesforce. The top half is the send phase; the bottom half is the completion phase that runs after the customer signs.



Technical sequence diagram: PDF creation through to signed document stored in Salesforce.

Simplified flow

The same journey told for a non-technical audience, useful for explaining the process to stakeholders who care about the outcome rather than the mechanics.



Stakeholder-friendly view of the signing journey.

Key design decisions

A handful of choices keep the pipeline automated, secure, and consistent with existing BRYT patterns.

DECISION	CHOICE	WHY
DocuSign authentication	JWT Grant (server-to-server)	No human login needed; the token can be cached and refreshed for a fully automated pipeline
Trigger	S3 event on Estimate 1's output bucket	Direct hook into the existing render pipeline; no intermediate queue required
Status notifications	DocuSign Connect webhook (per-envelope)	Real-time updates instead of polling; per-envelope config avoids account-level admin setup
Signed document storage	S3 and Salesforce	S3 as the durable system of record; Salesforce for day-to-day business access
Salesforce integration	OAuth, existing credential pattern	Reuses the established salesforceOauthKey / salesforceOauthSecret secrets already in place
Resilience	Exponential backoff, 3 attempts	DocuSign and Salesforce calls can fail transiently; retries avoid losing a signed document
Webhook security	HMAC signature validation	The endpoint must be public for DocuSign to reach it, so every request is verified as genuine

The two Lambdas in detail

Send Envelope Lambda. Triggered by the S3 event, it reads the metadata sidecar to find the customer's Salesforce reference, queries Salesforce for their name and email, authenticates to DocuSign, then creates an envelope containing the PDF with a signature field and the customer as the sole signer. Setting the envelope to sent makes DocuSign email the customer immediately. It records the envelope details in DynamoDB for traceability.

Webhook Lambda. Exposed via a public API Gateway endpoint, it receives DocuSign's callbacks. Every request is HMAC-validated first. On a completed event it downloads the

signed PDF, stores it in S3, attaches it to the Salesforce record, and updates the status. On a declined or expired event it records the status and writes a notification for follow-up.

External integrations

The pipeline talks to two external services. Both authenticate with credentials held in AWS Secrets Manager.

SERVICE	WHAT WE CALL IT FOR	KEY OPERATIONS
DocuSign eSignature API	Sending documents for signature and retrieving signed copies	JWT auth; create + send envelope; download combined signed document
Salesforce REST API	Finding the customer and filing the signed contract	OAuth; query contact by reference; upload file (ContentVersion); link to record

What we track

A single DynamoDB table records one row per envelope, for debugging and traceability. There is no user-facing status screen in this phase.

FIELD	HOLDS
Envelope ID	DocuSign's identifier for the signing envelope (the primary key)
Salesforce reference	Links back to the customer record (also a lookup index)
Contract note S3 key	The original PDF that was sent
Customer name & email	Who the envelope was sent to
Status	sent, completed, declined, or expired
Signed PDF S3 key	Where the signed copy landed, once complete
Timestamps & error/decline reason	When it was created and last updated, plus any failure detail

Error handling and resilience

Because this pipeline produces legally significant documents, it fails safe. Any failure in the send phase (missing metadata, customer not found, no email on file, DocuSign or Salesforce errors) is logged to a shared error bucket and halts before an envelope is created, so nothing is half-sent.

In the completion phase, the two external calls that matter most, downloading the signed PDF and uploading it to Salesforce, each retry up to three times with exponential backoff. If they still fail, an error record is written for manual investigation, and the signed PDF is always safe in S3 regardless.

Open questions

This estimate carries assumptions that need BRYT confirmation. Each has a working assumption so delivery can proceed, but they should be resolved before or during implementation.

#	QUESTION	WORKING ASSUMPTION
1	Are there multi-party signing scenarios (BRYT rep, TPI), or always just the customer?	Single signatory (customer only). Multi-party would affect envelope routing and signing order.
2	Does BRYT have an existing DocuSign account?	Starting from scratch, new account, API credentials, and sandbox needed. No evidence found in AWS.
3	Is DocuSign's standard branded email acceptable, or is full control needed?	DocuSign sends the signing email directly, with BRYT branding configured in DocuSign settings.
4	Is an Admin Portal UI needed for envelope status tracking?	No UI this phase. Metadata is stored for debugging; the signed PDF in Salesforce is the visible outcome.
5	What Salesforce object does the customer reference map to?	Resolved at implementation time; determines where the signed PDF attaches. Does not affect architecture.
6	Are voided envelopes, resend, and reminders out of scope?	Out of scope. We handle completed, declined, and expired only. Retry/resend can come in a later phase.

Delivery breakdown

Estimate 2 is ~4.1 required days plus optional testing, ~7.1 days in total. Work is grouped as follows.

AREA	SCOPE
Infrastructure & utilities	DynamoDB table + GSI, signed-docs S3 bucket, API Gateway webhook route, Secrets Manager, IAM, retry + error-record utilities
Salesforce client	OAuth auth, customer contact lookup, signed document upload (ContentVersion + link)

AREA	SCOPE
DocuSign client	JWT auth, envelope creation + send, signed document download, HMAC webhook validation
Metadata service	DynamoDB create / get / update / query-by-reference for envelope records
Send Envelope Lambda	S3 event handling, metadata extraction, and orchestration of the send phase
Webhook Lambda	HMAC validation, completion flow, declined/expired notification flow
Integration	CDK wiring, S3 event trigger, metadata sidecar in the render pipeline, end-to-end validation

Testing note

The build is designed around 10 correctness properties (trigger-to-envelope correlation, webhook validation, completion produces a signed PDF in both S3 and Salesforce, and fail-safe behaviour). Property-based and integration tests are marked optional in the plan and can be deferred for a faster MVP, but they are strongly recommended given the pipeline handles signed contracts.